

CSE 3025 Lab 3 Report

Part 1:

```
$ ps
name      pid      state    priority
init       1        SLEEPING      10
sh         2        SLEEPING      10
ps         3        RUNNING       10
$
```

Part 2:

```
$ lab3test 4 &
$ Parent C2hild1 c22 crreaeatiteng d
child 22
ParenCt 21hil cdre a23ti cng reactehidld
23
Parent 21 creating child 24
Parent 21 creating child 25
Child 24 created
ps
name      pid      state    priority
init       1        SLEEPING      10
sh         2        SLEEPING      10
lab3test   22        RUNNING       20
ps         26        RUNNING       10
lab3test   23        RUNNABLE      20
lab3test   24        RUNNING       20
lab3test   25        RUNNABLE      20
$ nice 5 25
Unrecognized process...
$ nice 25 5
Child 25 created
$ ps
name      pid      state    priority
init       1        SLEEPING      10
sh         2        SLEEPING      10
lab3test   22        RUNNING       20
ps         29        RUNNING       10
lab3test   23        RUNNABLE      20
lab3test   24        RUNNABLE      20
lab3test   25        RUNNING       5
$
```

Part 3:

As can be seen with the screenshot below, the processes do seem to be rotating mainly between “runnable” to “running”, indicative of the Round Robin scheduling working.

```
$ ps
name      pid      state  priority
init       1      SLEEPING    10
sh         2      SLEEPING    10
lab3test2  9        RUNNING    20
lab3test2  8        SLEEPING    10
lab3test2  5        SLEEPING    10
lab3test2  7        SLEEPING    10
lab3test2  10       RUNNABLE    20
lab3test2  11       RUNNING    20
ps         12      RUNNING    10
$ ps
name      pid      state  priority
init       1      SLEEPING    10
sh         2      SLEEPING    10
lab3test2  9        RUNNABLE    20
lab3test2  8        SLEEPING    10
lab3test2  5        SLEEPING    10
lab3test2  7        SLEEPING    10
lab3test2  10       RUNNING    20
lab3test2  11       RUNNING    20
ps         13      RUNNING    10
$ ps
name      pid      state  priority
init       1      SLEEPING    10
sh         2      SLEEPING    10
lab3test2  9        RUNNABLE    20
lab3test2  8        SLEEPING    10
lab3test2  5        SLEEPING    10
lab3test2  7        SLEEPING    10
lab3test2  10       RUNNING    20
lab3test2  11       RUNNING    20
ps         14      RUNNING    10
$ ps
name      pid      state  priority
init       1      SLEEPING    10
sh         2      SLEEPING    10
lab3test2  9        RUNNING    20
lab3test2  8        SLEEPING    10
lab3test2  5        SLEEPING    10
lab3test2  7        SLEEPING    10
lab3test2  10       RUNNABLE    20
lab3test2  11       RUNNING    20
ps         15      RUNNING    10
$ ps
name      pid      state  priority
init       1      SLEEPING    10
sh         2      SLEEPING    10
lab3test2  9        RUNNING    20
```

Part 4:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ lab3test2 & lab3test2 & lab3test2 &
$ ParCenht 5 crild ea9 createPd
arent 8 creatingP archent il7 dcre 10ati
ntg inchg ilchd illd 9

CChihild ld 1110 cr ceatreedate
d

$ ps
Child 10 terminated
Child 11 terminated
name      pid      state  priority
init       1        RUNNING  10
sh         2        SLEEPING  10
lab3test2   9        RUNNING  20
lab3test2   5        SLEEPING  10
ps         12        RUNNING  10
C$ hild 9 terminated

$ ps
name      pid      state  priority
init       1        SLEEPING  10
sh         2        SLEEPING  10
ps         13        RUNNING  10
```

This output shows that priority scheduling works because the ps command had a priority of 10 and so did the parent processes of the lab3test. So when we ran the test and then ran ps it waited until the parent processes were finished utilizing a first come first serve scheduling due to the processes having the same priority before outputting the current processes and their statuses. It then showed the child processes running which again aligns with the priority scheduling we implemented because the child processes had a priority of 20 which meant that they would only run after ps and the parent processes would be finished. This is then highlighted by the output we see above.